
Clase 072 — Curvas de aprendizaje y bias-variance tradeoff

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4. Duración estimada: 60 min.

Clase 072 — Curvas de aprendizaje y bias-variance tradeoff

Parte: 1 — Machine Learning Clásico · Fuente: Géron, cap. 4. Duración estimada: 60 min.

Objetivo

Diagnosticar si un modelo sufre de alto sesgo o alta varianza leyendo curvas de aprendizaje (`sklearn.model_selection.learning_curve`) y decidir, con criterio, si conviene conseguir más datos, aumentar la capacidad del modelo o regularizar.

Resultados de aprendizaje

Al finalizar, el estudiante podrá:

1. Graficar una curva de aprendizaje (RMSE train vs. RMSE validación en función de `train_size`) con `learning_curve`.
2. Identificar patrones canónicos: underfitting (curvas altas y juntas) vs. overfitting (gap persistente).
3. Descomponer conceptualmente el error esperado en $\text{bias}^2 + \text{variance} + \text{irreducible noise}$.
4. Decidir acción correctiva apropiada: más datos, más capacidad, features nuevas, o regularización.
5. Justificar por qué "más datos" no siempre es la solución (caso de high bias).

Temas

- Curva de aprendizaje: qué se plotea y cómo se lee.
- Patrón de underfitting: ambas curvas convergen alto → más datos no ayuda.
- Patrón de overfitting: gap grande train/val → más datos sí ayuda, o regularizar.
- Descomposición bias-variance del error de generalización.
- Error irreducible (ruido de Bayes): cota inferior inevitable.
- Tradeoff: aumentar capacidad ↓ bias pero ↑ variance.
- Diagnóstico operacional con `learning_curve` y `validation_curve`.

Definiciones y características

- Learning curve (curva de aprendizaje) — gráfico del error de entrenamiento y validación en función del tamaño del conjunto de entrenamiento. Permite diagnosticar capacidad vs. datos.
- Bias (sesgo) — error por supuestos erróneos del modelo (p. ej., asumir lineal lo no-lineal). Modelos con alto sesgo subajustan.
- Variance (varianza) — sensibilidad del modelo a pequeñas variaciones en los datos de entrenamiento. Modelos con alta varianza sobreajustan.
- Irreducible error — ruido intrínseco de los datos ($\text{Var}(\epsilon)$); ninguna mejora de modelo lo elimina. Es la cota inferior.
- Diagnóstico high-bias — `train_error` alto y `val_error` alto, ambas curvas convergen a un valor alto cuando `m` crece. Síntoma: más datos no mueven la aguja.
- Diagnóstico high-variance — `train_error` bajo, `val_error` notablemente más alto, gap persistente. Síntoma: el modelo memoriza; más datos o regularización deberían cerrar el gap.
- Bias-variance tradeoff — $E[(y - \hat{y})^2] = \text{Bias}^2 + \text{Var} + \sigma^2$. Reducir uno suele aumentar el otro; el óptimo

está en el medio.

- Capacidad del modelo — grados de libertad efectivos (grado del polinomio, profundidad del árbol, n° de parámetros). Más capacidad menos bias, más variance.

Dataset / recursos

- Dataset sintético tipo "noisy quadratic": $y = 0.5 \cdot x^2 + x + 2 + \epsilon$ con `np.random.randn`. Permite controlar la complejidad
- `sklearn.datasets.fetch_california_housing (subset)` para una corrida sobre datos reales.
- API clave: `sklearn.model_selection.learning_curve`, `validation_curve`.

Ejercicios

1. Curva base. Generá 200 puntos del dataset cuadrático ruidoso. Ajustá una regresión lineal y graficá la curva de aprendiz
1. Aumentar capacidad. Repetí con `PolynomialFeatures(degree=2) + LinearRegression`. Comparalo con `degree=10`. Identificá cu
1. ¿Más datos ayudan? Para el polinomio de grado 10, extendé el dataset a 2000 puntos y volvé a plotear. ¿Se cierra e
1. Validation curve. Usá `validation_curve` para barrer `degree` de 1 a 15 sobre el mismo dataset. Encontrá el "sweet spot
1. Bias-variance empírico. Entrená 100 modelos `degree=10` sobre bootstraps del dataset y calculá, para una grilla de x de test

$\text{bias}^2 + \text{var}$ se acerca al MSE total menos σ^2 .

Homework verifiable

Sobre el dataset `california_housing`:

1. Entrená `DecisionTreeRegressor(max_depth=d)` para `d` {2, 5, 10, None}.
2. Para cada uno, generá la curva de aprendizaje con `learning_curve(cv=5, scoring='neg_root_mean_squared_error')`.
3. Clasificá cada modelo como underfit, fit razonable o overfit justificando con el gap final y el nivel de error.
1. Recomendá explícitamente, para cada caso, una de tres acciones: ["más datos", "más capacidad", "regularizar/podar"].

Criterio de aceptación: un .py o notebook que, al ejecutarse, imprima una tabla con columnas

`depth | train_rmse_final | val_rmse_final | gap | diagnostico | accion_recomendada` y guarde los 4 gráficos en `figs/learning_curve_depth_{d}.png`.

Errores comunes

- Confundir curva de aprendizaje con validation curve. La primera varía m (tamaño de train); la segunda varía un hiperparámetro.
- Leer el error de train absoluto como diagnóstico. Lo relevante es el gap y la tendencia asintótica, no un punto aislado.
- Recomendar "más datos" frente a high bias. Si ambas curvas ya convergieron alto, sumar filas no baja el error; hay que cambiar el modelo.
- No promediar sobre CV. Una sola partición es ruidosa; `learning_curve` ya hace CV interno — usalo.
- Olvidar el irreducible error. Apuntar a $\text{val_rmse} = 0$ es absurdo si σ del ruido es positivo; siempre hay piso.

Preguntas frecuentes

- ¿Más datos o modelo más complejo? Mirá el gap. Gap grande con train bajo → overfit → más datos o regularización ayudan. Gap pequeño → más datos no ayudan.
- ¿Cuántos puntos uso para `train_sizes`? Por defecto `np.linspace(0.1, 1.0, 10)` está bien. Para datasets grandes, usá menos puntos.
- ¿Por qué el `train_error` sube al aumentar m ? Con pocos datos el modelo memoriza ($\text{train_error} \approx 0$); al sumar más datos, el error sube.
- ¿Sirve para clasificación? Sí. Usá `scoring='accuracy'` o `'neg_log_loss'`; la interpretación del gap es análoga.
- ¿Bias-variance se mide en la práctica? Conceptualmente sí (vía bootstrap, como el ejercicio 5), pero en producción no se suele usar.

Referencias

- Géron, A. Hands-On Machine Learning, 3ª ed., cap. 4 — "Learning Curves" y cap. 7 — "Bias/Variance Tradeoff".
- scikit-learn user guide: Validation curves: plotting scores to evaluate models.
- API: `sklearn.model_selection.learning_curve`.

Material descargable

- Guía explicativa (PDF) — versión imprimible con todo el contenido de la clase.
- Presentación (PPTX) — deck PowerPoint listo para proyectar en clase.
- Notebook ejecutable (.ipynb) — ábralo desde el laboratorio del programa o desde Jupyter.

Siguiente clase

Clase 073 — Regularización: Ridge, Lasso, Elastic Net

Apéndice: notebook (primer bloque)

$y = 0.5x^2 + x + 2 + \epsilon$, con $\epsilon \sim \mathcal{N}(0, \sigma^2)$ y $\sigma=1$. Conocer σ nos permite comparar el error irreducible con la varianza medida.

```
import numpy as np
import matplotlib.pyplot as plt

np.random.seed(42)
m = 200
X = 6 * np.random.rand(m, 1) - 3
sigma = 1.0
y = 0.5 * X[:, 0]**2 + X[:, 0] + 2 + np.random.randn(m) * sigma
print('dataset:', X.shape, '| ruido  $\sigma$  =', sigma, '|  $\sigma^2$  =', sigma**2)
```

Archivos complementarios

- notebook.ipynb